

Кафедра
статистики и
кибернетики
*Department of statistics and
cybernetics*

Лекция №1

«Введение в алгоритмы и структуры данных»

Лектор: Вадим Владимирович Демичев
доцент, кандидат экономических наук

Москва, 2026



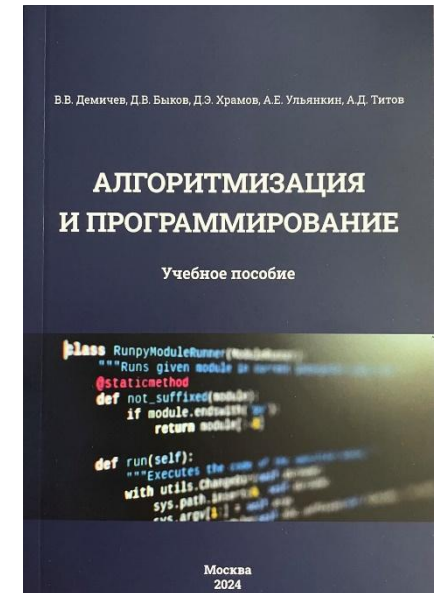
Трофимов,
В. В. Алгоритмизация и
программирование : учебник
для вузов / В. В. Трофимов,
Т. А. Павловская ; под
редакцией В. В. Трофимова. —
Москва : Издательство Юрайт,
2022. — 137 с.



Грокаем алгоритмы.
Иллюстрированное
пособие для
программистов и
любопытствующих
Книга, Адитья Бхаргав



Искусство
программирования
Книга Дональда Кнута



Алгоритмизация и
программирование:
учебное пособие / В.В.
Демичев, Д.В. Быков,
Д.Э. Храмов, А.Е.
Ульянкин, А.Д. Титов. —
М.: Издательство
«Научныйконсультант»,
2024. – 248 с.

Алгоритм:

- Алгоритмом называется набор инструкций для выполнения некоторой задачи
- Любой фрагмент кода можно назвать алгоритмом
- Алгоритм это инструмент для решения конкретно поставленной задачи

Структуры данных:

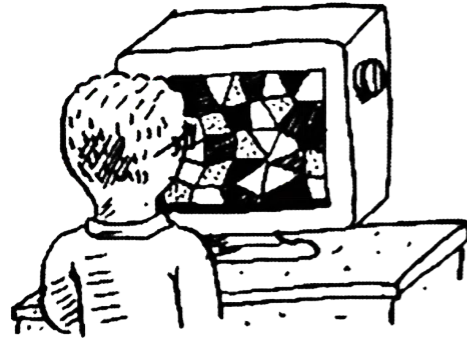
- Структура данных - это способ хранения и организации данных, облегчающий доступ к этим данным и их модификацию.
- Ни одна структура не является универсальной и не может подходить для всех целей, поэтому важно знать преимущества и ограничения, присущие некоторые из них.

Что мы должны понять прежде всего?:

- Реализация каждого алгоритма, который мы рассмотрим уже доступна практически на любом языке программирования.
- Любая реализация будет бесполезна, если вы не понимаете ее плюсов и минусов.
- Рассмотрим сильные и слабые стороны разных алгоритмов

Бинарный поиск

- Найти номер человека в телефонной книге
- Проверка учетной записи [Вконтакте](#)



Определение бинарного поиска:

- Бинарный поиск - это алгоритм; на входе он получает отсортированный список элементов.
- Если элемент, который вы ищете, присутствует в списке, то бинарный поиск возвращает ту позицию, в которой он был найден. В противном случае бинарный поиск возвращает null.

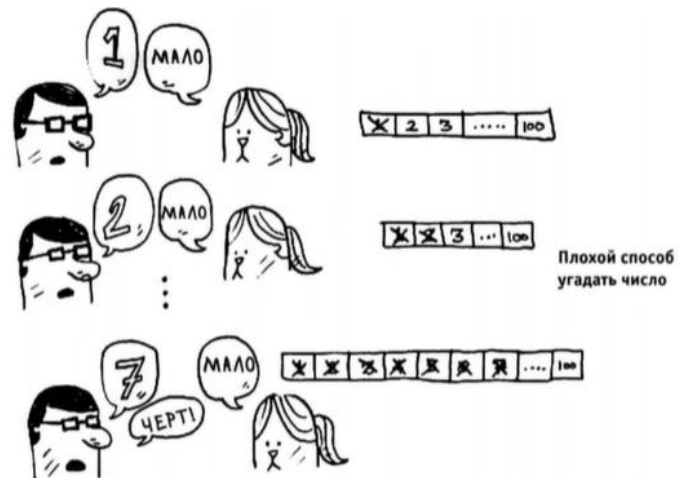
Игра:

- Я загадал число от 1 до 100

1	2	3	...	100
---	---	---	-----	-----

- Вы должны отгадать мое число, используя как можно меньше попыток. При каждой попытке я буду давать один из трех ответов: «мало», «много» или «угадал».

Перебор:



Это пример простого поиска. При каждой догадке исключается только одно число. Если я загадал число 99, то , чтобы добраться до него. потребуется 99 попыток!

Более эффективный поиск:

- Начнем с 50

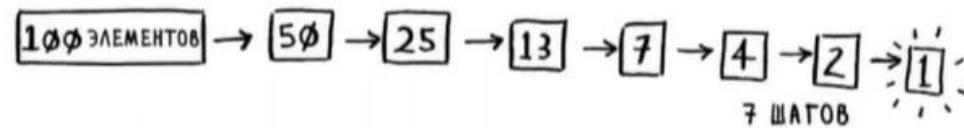


- Слишком мало ... но вы только что исключили половину чисел!
Теперь вы знаете, что все числа 1-50 меньше загаданного.
Следующая попытка: 75.



Максимальное количество шагов:

- При бинарном поиске каждый раз исключается половина чисел



- Какое бы число ни было задумал, вы гарантированно сможете угадать его не более чем за 7 попыток, потому что с каждой попыткой исключается половина оставшихся чисел!

Вопрос:

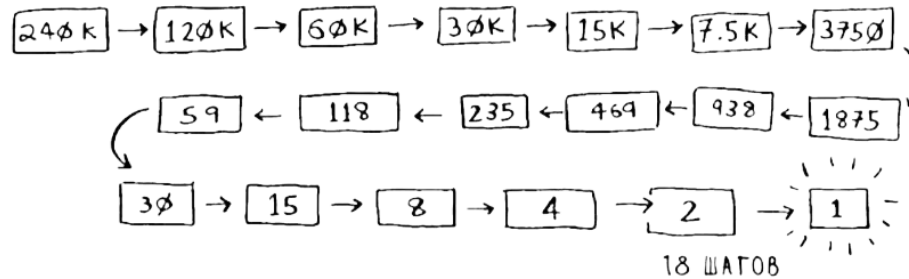
- Предположим, вы ищете слово в словаре с 240 000 словами. Как вы думаете, сколько попыток вам понадобится в худшем случае?

ПРОСТОЙ ПОИСК: ____ ШАГОВ

БИНАРНЫЙ ПОИСК: ____ ШАГОВ

Ответ:

- При простом поиске может потребоваться 240 000 попыток, если искомое слово находится на самой последней позиции в книге.
- С каждым шагом бинарного поиска количество слов сокращается вдвое, пока не останется только одно слово. Итак, бинарный поиск потребует 18 шагов - заметная разница!



Вычисление количества шагов:

- В общем случае для списка из n элементов бинарный поиск выполняется за $\log_2 n$ шагов, тогда как простой поиск будет выполнен за n шагов.

$$10^2 = 100 \leftrightarrow \log_{10} 100 = 2$$

$$10^3 = 1000 \leftrightarrow \log_{10} 1000 = 3$$

$$2^3 = 8 \leftrightarrow \log_2 8 = 3$$

$$2^4 = 16 \leftrightarrow \log_2 16 = 4$$

$$2^5 = 32 \leftrightarrow \log_2 32 = 5$$

Ограничение алгоритма:

- Бинарный поиск работает только в том случае, если список **отсортирован**.
- Например, имена в телефонной книге хранятся в алфавитном порядке, и вы можете воспользоваться бинарным поиском. А что произойдет, если имена не будут отсортированы?

Реализация бинарного поиска:

[9]
✓ 0 сек.

```
def binary_search(list, item):  
    left = 0  
    right = len(list) - 1  
  
    while left <= right:  
        mid = (left + right) // 2  
        if list[mid] == item:  
            return mid  
        if list[mid] < item:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return None
```

[13]
✓ 0 сек.

```
my_list = [1, 4, 6, 7, 12, 46]  
binary_search(my_list, 12)
```



... 4

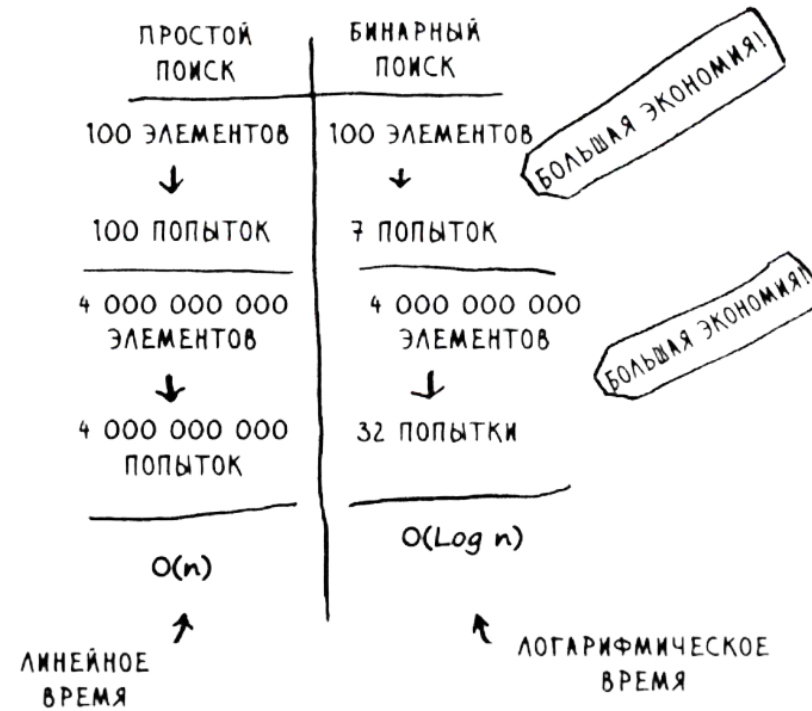
Время выполнения:

- В первом варианте мы последовательно проверяли каждое число, одно за другим. Если список состоит из 100 чисел, может потребоваться до 100 попыток. Для списка из 4 миллиардов чисел потребуются о 4 миллиардов попыток. Таким образом, максимальное количество попыток совпадает с размером списка.
- Такое время выполнения называется **линейным**.

Время выполнения:

- С бинарным поиском дело обстоит иначе. Если список состоит из 100 элементов, потребуется не более 7 попыток. Для списка из 4 миллиардов элементов потребуется не более 32 попыток.
- Бинарный поиск выполняется за **логарифмическое** время.

Сравнение времени выполнения:



О-большое:

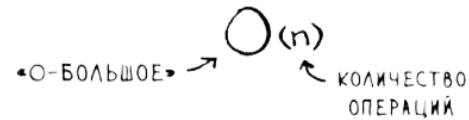
- Специальная нотация “О-большое” описывает скорость работы алгоритма.
- Время от времени вам придется использовать чужие алгоритмы, а потому неплохо было бы понимать, насколько быстро или медленно они работают.

	ПРОСТОЙ ПОИСК	БИНАРНЫЙ ПОИСК
100 ЭЛЕМЕНТОВ	100 мс	7 мс
10 000 ЭЛЕМЕНТОВ	10 секунд	14 мс
1 000 000 ЭЛЕМЕНТОВ	11 дней	32 мс

Время выполнения растет с совершенно разной скоростью!

Определение «О-большое»:

- "О-большое" описывает, насколько быстро работает алгоритм. Предположим, имеется список размера n . Простой поиск должен проверить каждый элемент, поэтому ему придется выполнить n операций. Время выполнения О-большое имеет вид $O(n)$.
- "О-большое" не сообщает скорость в секундах, а позволяет сравнить количество операций .
- Оно указывает, насколько быстро возрастает время выполнения алгоритма.



Как записывается «О-большое»

Определение «О-большое»:

- Сложность $O(1)$
- Сложность $O(n)$
- Сложность $O(\log(n))$
- Сложность $O(n \cdot \log(n))$
- Сложность $O(n^2)$

Сложность $O(2^N)$:

- Экспоненциальные функции, такие как 2^N , возрастают молниеносно и поэтому полезны для ограниченного круга задач.
- Обычно посредством алгоритмов с подобным временем работы ищется оптимальный набор входных данных

Задача о рюкзаке:

- Представьте, что вы жадный воришка. Вы забрались в магазин с рюкзаком, и перед вами множество товаров, которые вы можете украсть. Однако емкость рюкзака не бесконечна: он выдержит не более 35 фунтов.
- Требуется подобрать набор товаров максимальной стоимости, которые можно сложить в рюкзак. Какой алгоритм вы будете использовать?

Задача о рюкзаке - пример:



МАГНИТОФОН
\$ 3000
30 фунтов



НОУТБУК
\$ 2000
20 фунтов

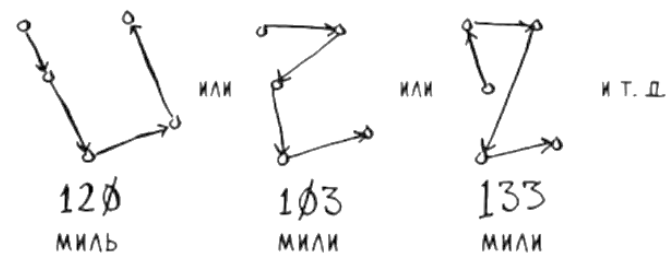


ГИТАРА
\$ 1500
15 фунтов

Сложность $O(N!)$:

- Возрастает намного быстрее, чем экспоненциальная функция 2^N .
- Оптимальное распределение входных данных

Задача коммивояжера:

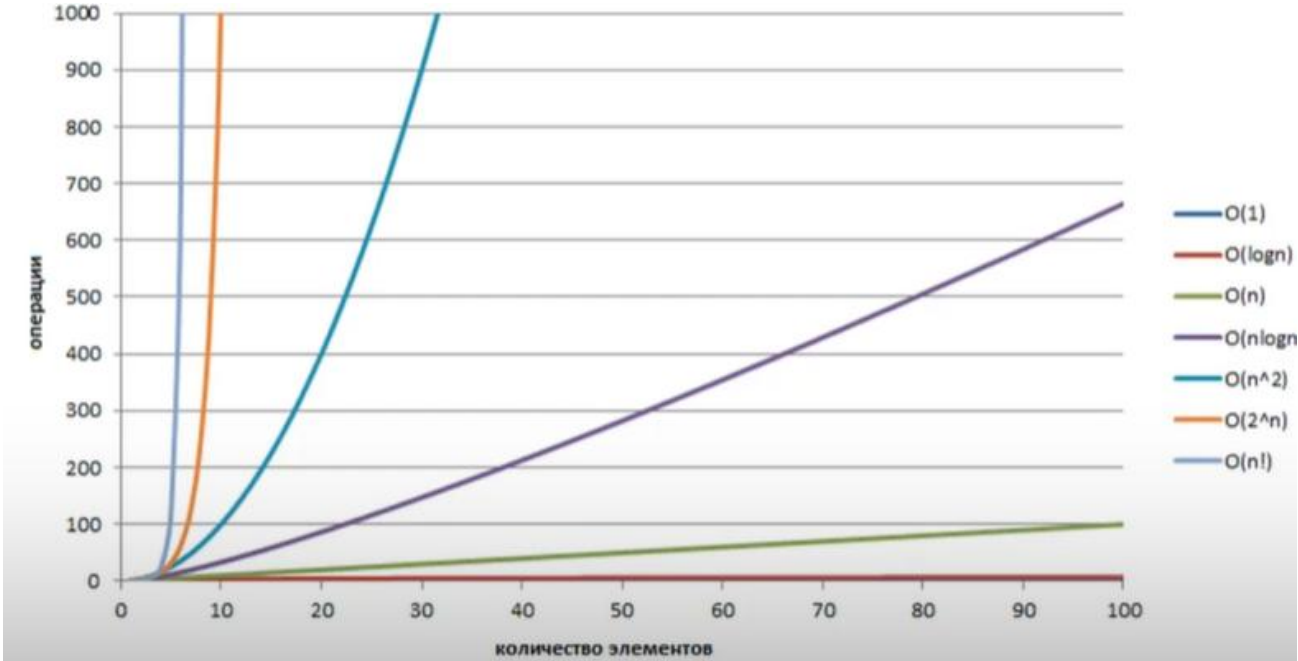


Задача коммивояжера - продолжение:

ГОРОДА	ОПЕРАЦИИ
6	720
7	5040
8	40320
...	...
15	1,307,674,368,000
...	...
30	2,652,528,512,11,068,36,308,480,000,000

Количество операций стремительно растет

График роста O-большое:



Сравнение сложности алгоритмов сортировки:

Алгоритм	Структура данных	Временная сложность		
		Лучшее	В среднем	В худшем
Быстрая сортировка	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$
Сортировка слиянием	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Пирамидальная сортировка	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Пузырьковая сортировка	Массив	$O(n)$	$O(n^2)$	$O(n^2)$
Сортировка вставками	Массив	$O(n)$	$O(n^2)$	$O(n^2)$
Сортировка выбором	Массив	$O(n^2)$	$O(n^2)$	$O(n^2)$
Блочная сортировка	Массив	$O(n+k)$	$O(n+k)$	$O(n^2)$
Поразрядная сортировка	Массив	$O(nk)$	$O(nk)$	$O(nk)$

Вопросы

